

PROGRAMACIÓN 1

Tipos de datos



Universidad Católica de La Plata
Facultad de Ciencias Exactas y Tecnología

Andrés Garavaglia
Antonio Restelli

Contenido

- Tipos de Datos
 - Definidos por el lenguaje
 - Enteros, Reales, Cadena de texto, Operaciones y Funciones
 - Definidos por el usuario
 - Subrango, Enumerados, Conjuntos
- Estructuras
 - Clasificación
 - Arreglos (Arrays)
 - Vectores
 - Matrices

Tipos de datos



A toda variable que se desee utilizar en un programa debe asignarse un tipo de dato específico.

Un tipo de dato define el posible rango de valores que una variable puede tomar al momento de ejecución del programa y a lo largo de toda la vida útil del programa.

- Tipos de datos:
 - **Definidos por el lenguaje de programación.**
 - **Definidos por el usuario.**

Definidos por el lenguaje.

- Enteros

Nombre	Rango (desde...hasta)	Tamaño (bytes)	Formato
Integer	-32.768 a 32.767	2	Entero con signo
Word	0 a 65535	2	Entero sin signo
ShortInt	-127 a 127	1	Entero corto con signo
Byte	0 a 255	1	Entero corto sin signo
LongInt	-2.147.483.648 a 2.147.483.647	4	Entero largo con signo

- Reales

Nombre	Rango (desde...hasta)	Tamaño (bytes)	Cifra significativas
Real	2,9E-39 a 1,7E38	6	11-12
Single	1,5E-45 a 3,4E38	4	6-7
Double	5,0E-307 a 1,7E307	8	15-16
Extended	1,9E-4932 a 1,1E493210	10	19-20
Comp	-9,2E18 a 9,2E18	8	18-19

Definidos por el lenguaje.

- Char

El tipo Char depende del código de caracteres empleado por la máquina. El más utilizado es el código ASCII. Una variable o constante tipo Char puede contener un solo carácter especificado entre apóstrofes. Ejem. 'a' 'M'

- Lógicos

Pueden tomar dos valores True (verdadero) False (falso). El identificador estándar boolean define una variable de este tipo.

- Cadenas

Una cadena (string) es una secuencia de caracteres que tiene una longitud máxima de 255 caracteres. Los caracteres que componen la cadena se delimitan con apóstrofes. Ejem.

'abcd' longitud de la cadena 4
" cadena vacía o nula.

Una variable de cadena se define utilizando la palabra reservada String y el tamaño físico máximo que pueda alcanzar durante la ejecución del programa.

Cadena de texto (string)

- Definición

```
Var
    MiNombre:string[30];
    MiApellido:string[50];

Begin
    MiNombre:='Blas';
    MiApellido:='Pascal';

end.
```

- Operaciones básicas:

- Concatenar (+)
- Comparar(=,>,<,<>)
- Longitud

Concat (X1, X2, Xn) – Donde x son cadenas de caracteres o variables string.

Ejemplo: texto:= **concat** ('hola ', 'Mundo');

Length (X) – Donde X es una cadena de texto o variable string

Ejemplo: longitud:= **length** ('Programacion');

Cadena de texto (string)

- Funciones de tratamiento de cadenas

Instrucción	Función	Instrucción	Función
COPY	Extrae una subcadena de caracteres de otra cadena de caracteres Copy (cadl,po,num) Po: primera posición del caracter a extraer. Num: número de caracteres que se extraen. Po y Num deben ser enteros	INSERT	Inserta una cadena de caracteres en otra a partir de una posición dada. Insert (subcadena,destino,posicion) Subcadena: Cadena a insertar Destino: Cadena donde se va a insertar Posicion: Lugar a partir del cual se va a insertar
POS	Determina si una cadena es subcadena de otra, en caso afirmativo devuelve la posición donde comienza la subcadena, en caso negativo devolvería cero. Pos (subcadena,cadena)	UPCASE	Devuelve el caracter mayúscula Uppcase(x) Donde x es una variable de tipo char.
DELETE	Suprime el número de caracteres que le digamos de una cadena a partir de la posición que le indiquemos Delete (Cad,Po,Num)	STR	Convierte un valor numerico a la correspondiente cadena de caracteres que lo representa. Str(valor,cad)
VAL	Procedimiento inverso a Str, es decir, devuelve el valor numerico de una cadena. Val(cad,variable,codigo) Cad: la cadena a convertir en valor numerico. Variable: el numero que se obtenga. Codigo: cero si se ha podido convertir.		

StrToInt (X) – Donde x es una cadena de caracteres o variable string.

Ejemplo: numero:= strtoint('1234');

IntToStr (N) – Donde n es un entero o una variable int

Ejemplo: texto:= inttostr('4321');

Exactamente igual para StrToFloat y FloatToStr

Definidos por el usuario



Un tipo de dato define el posible rango de valores que una variable puede tomar al momento de ejecución del programa y a lo largo de toda la vida útil del programa. Este rango esta definido por el usuario.

- Tipos de datos definidos por el usuario:
 - **Del tipo simple**
 - Subrango
 - Enumerados
 - **Del tipo estructurado**
 - Conjunto

Tipo Subrango

- Se define a partir de un ordinal estableciendo los límites superiores e inferiores, los subrangos se utilizan para darle una mayor legibilidad a los programas.

```
Type      Subrango= lim_inf..lim_sup;  
Var  
    s:Subrango
```

```
Type      Meses= 1..12;  
Var  
    mes:Meses;
```

```
Type      Dias= 1..7;  
Var  
    dia:Dias;
```

Tipo Enumerado

- Han de ser del tipo ordinal. Se establecen los valores que van a poder tomar las variables mediante la especificación de una lista de identificadores válidos. Los identificadores han de comenzar con un carácter y no se pueden repetir.
No es posible leer desde teclado ni imprimir directamente sobre la variable.

Type

Dias= (Lunes, Martes, Miercoles, Jueves, Viernes, Sabado, Domingo)

Var

Dia:Dias;

Begin

dia:= Jueves;

Ord(dia); // 3

Pred(dia) // Miercoles

Succ(dia) // Viernes

End.

Operaciones que admite :

- de relación (>,<,<>,>=,<=,...)
- de asignación
- Ord (e) ;
- Pred (e) ;
- Succ (e) ;

Tipo Conjunto



Un conjunto es una colección de objetos relacionados. Cada elemento dentro del conjunto es llamado miembro o elemento del conjunto.

- Aunque en matemáticas no hay restricciones en pascal los objetos:
 - Los elementos de un conjunto deben ser del mismo tipo, llamado tipo base.
 - El tipo base debe ser un tipo simple, excepto el real.

```
Type  
dias_mes = set of 0..31;  
mayusculas = set of 'A'..'Z';  
caracteres = set of char;  
equipos = (Boca, River, Racing, Independiente);  
futbol = set of equipos;  
Var  
GrupoA,GrupoB : futbol;
```

Tipo Conjunto

Operaciones sobre dos conjuntos A y B		
Operación	Notación Pascal	Conjunto resultante
Unión	+	A+B es el conjunto que contiene todos los elementos que están en A, en B o en ambos.
Intersección	*	A*B es el conjunto cuyos elementos pertenecen a A y B simultáneamente.
Diferencia	-	A-B es el conjunto cuyos elementos son de A pero no de B.

Operadores de relación de conjuntos A y B		
Operador	Nombre del operador	Resultado
$\leq$	Subconjunto	El valor de $A \leq B$ es true. Si cada elemento de A es también de B. En caso contrario es false.
=	Igualdad	El valor de $A = B$ es true si cada elemento de A está en B y cada elemento de B está en A. En caso contrario $A = B$ es falso, $A=B$ equivale a $(A \leq B)$ and $(B \leq A)$.
$\neq$	Desigualdad	El valor de $A \neq B$ es true si el valor de $A = B$ es false y viceversa.
$\geq$	Superconjunto	$A \neq B$ equivale a $\text{not } (A = B)$, $A \geq B$ es true si $B \leq A$ es true.

Estructuras

Clasificación:

- **Respecto al número de componentes.**
 - **Estáticas:** el número de componentes es fijo durante la ejecución del programa.
 - **Dinámicas:** el número de componentes varia durante la ejecución del programa.
- **Respecto al tipo de los componentes.**
 - **Por combinación de datos de diferentes tipos** (registros).
 - **Por repetición de datos del mismo tipo** (arrays, conjuntos, archivos, listas, árboles, etc.).

Arreglos (Arrays)



Es una estructura homogénea de datos de tamaño constante. La forma de acceder a cada uno de sus elementos es a través de un identificador común y uno o varios índices.

- Características:
 - Todos los elementos del array son del mismo tipo.
 - El número de ellos no varía durante la ejecución del programa.
 - El número máximo de valores posibles que pueda tomar cada índice se denomina rango de esa dimensión.
 - Los valores han de ser consecutivos, por lo que el índice es un ordinal.
 - Se accede a la estructura mediante un identificador común (el nombre del array) y un índice.

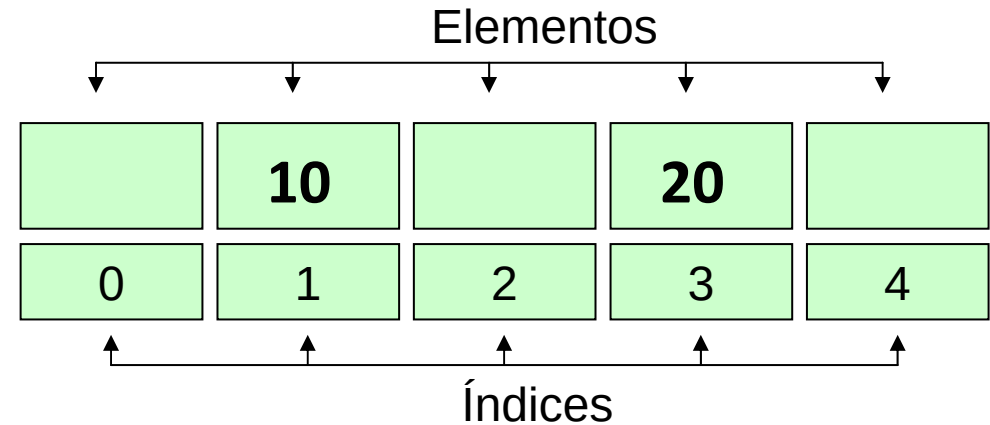
Arreglos (Arrays) una dimensión

Type

```
vector = array[1..5] of integer;
```

Var

```
Mivector: vector;
```



```
Mivector[1]:= 10;
```

```
Mivector[3]:= 20;
```

```
Longitud:= length (Mivector) // Cantidad de elementos del vector (5)
```

Recorrer un vector

```
For i:=0 to Length (Mivector) - 1 do  
  Writeln (Mivector[i]);
```

Arreglos (Arrays) dos dimensiones

```
Type  
    matriz = array[1..4,1..4] of integer;  
Var  
    Mimatriz: matriz;
```

```
Mimatriz[0][0]:= 1;
```

```
Mimatriz[1][1]:= 1;
```

```
Mimatriz[2][2]:= 1;
```

```
Mimatriz[3][3]:= 1;
```

Índices	0	1	2	3
0	1			
1		1		
2			1	
3				1

Recorrer una matriz

```
fila:=4;  
columna:=4;  
for i:=0 to fila do  
    for j:=0 to columna do  
        Writeln (Mimatriz[i][j]);
```

Nota: el orden de declaración de la matriz es fila y luego columna

Arreglos vectores

```
Type
    texto = array[1..10] of char;
Var
    t: texto;
```

```
T[0]:='H';
T[1]:='O';
T[2]:='L';
T[3]:='A';
```

```
for i:=0 to length(t) do
    Writeln (t[i]);
```

```
Var
    t: string;
Begin
    t:='HOLA';
End.
```

```
for i:=0 to length(t) do
    Writeln (t[i]);
```

Nota: las cadenas de texto (string) se manejan como si fueran vectores.
Podría decir que son un vector de caracteres (char)

Practica para la clase

1. **Escriba un programa que almacene en un vector los días de la semana.**
2. **Escriba un programa que almacene en un vector los números pares del 1 al 10.**
3. **Escriba un programa que lea las dimensiones de una matriz, visualice la matriz y que encuentre el menor y mayor elemento de esa matriz y sus posiciones.**
4. **Escribir un programa que en una matriz almacene los números primos del 1 al 100.**